

Restaurant Management System (RMS)

Prepared: 28 April 2026

1. Executive Summary

A full-stack web application that unifies restaurant operations into a single platform — covering the kitchen, front of house, purchasing, and creative development. The system eliminates manual processes, reduces waste, and gives management real-time visibility across all departments.

Built on Next.js 15, PostgreSQL, and the Claude AI API, the RMS is designed for modern restaurant groups that want to move from fragmented spreadsheets and paper processes to a unified, data-driven operation.

2. Core Modules

#	Module	Purpose
1	Recipe Management	Central recipe library with costing, versioning & allergens
2	POS (Point of Sale)	Table orders, payments, splits, KDS integration
3	Kitchen Stock Mgmt	Ingredient inventory, consumption tracking, wastage
4	FOH Stock Mgmt	Beverages, packaging & front-of-house supplies
5	Supplier Auto-Order	Smart reorder triggers based on par levels
6	Kitchen Lab (AI)	AI-assisted new menu creation with chef collaboration
7	Analytics & Reports	Costs, sales, waste, supplier performance dashboards
8	Staff & Role Mgmt	Roles: Admin, Manager, Chef, Sous-Chef, Waiter, Bartender

3. Goals & Success Metrics

Goal	Target Metric
Reduce stock waste	Wastage cost < 3% of revenue
Eliminate stockouts	Zero emergency orders per month
Speed up table ordering	Table-to-kitchen ticket < 30 seconds
Empower culinary creativity	New menu item prototype cycle < 1 day

Financial visibility

Real-time food cost % per dish

4. Technology Stack

Frontend

Layer	Technology	Reason
Framework	Next.js 15 (App Router)	SSR for speed, API routes built-in
Language	TypeScript	Full-stack type safety
UI Library	shadcn/ui + Tailwind CSS	Fast, customizable, accessible
State	Zustand	Lightweight, suits POS real-time state
Real-time	Socket.io	POS !' KDS live ticket updates
Forms	React Hook Form + Zod	Shared validation with backend

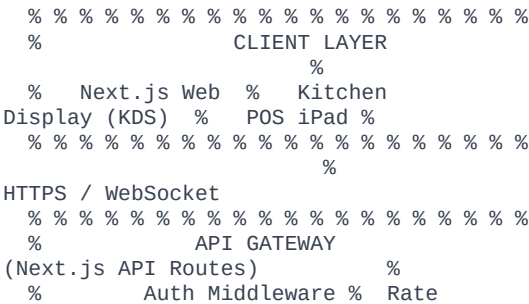
Backend & Database

Layer	Technology	Reason
Runtime	Node.js (Next.js API)	Unified monorepo codebase
ORM	Prisma 7	Type-safe DB access, clean migrations
Auth	NextAuth v5	JWT sessions, role-based access
Queue	BullMQ + Redis	Auto-ordering jobs, async tasks
Primary DB	PostgreSQL 17	ACID, relational, perfect for POS/ inventory
Cache	Redis	Sessions, real-time POS state, job queue
AI	Claude API (Sonnet 4.6)	Kitchen Lab recipe generation & analysis

5. System Architecture

Modular Monolith (Microservices-ready)

The system starts as a well-structured monolith — faster to build and easier to debug — with each module isolated into its own service layer so it can be extracted into an independent microservice as the product scales.



Limiting % Logging %
% % % %, % % % % % % % %, % % % % %
% % % % %
% % % % % % % % % % % % % % %
% % % % % % % % % % % % % % %
% POS % % Stock % % Recipe % % AI
Lab %
% Svc % % Svc % % Svc % %
(Claude) %
% % % %, % % % % %, % % % %, %
% % % % % % % % % % % % % % %
%
% % % % % % % % % % % % % % %
% PostgreSQL % Redis %
% % % % % % % % % % % % % % %

6. Database Schema (Key Entities)

users

Staff accounts with roles (ADMIN, MANAGER, CHEF, SOUS_CHEF, WAITER, BARTENDER)

ingredients

Ingredient library with unit, cost, stock level, par level, reorder point

allergens

Allergen master list linked to both ingredients and recipes

recipes

Recipe cards with category, prep/cook time, servings, selling price

recipe_ingredients

Junction: recipe !' ingredient with quantity and unit

recipe_allergens

Junction: recipe !' allergen (auto-derived + manual)

recipe_versions

Full JSON snapshot saved every time a recipe is edited

tables

Restaurant floor plan — section, capacity, status

orders

POS orders linked to table, server, open/close timestamps

order_items

Individual line items with status (PENDING!'COOKING!'READY!'SERVED)

menu_items

Published menu items linked to recipes with live pricing

foh_items

FOH inventory: beverages, packaging, front supplies

stock_transactions

Audit log of every stock movement (IN / OUT / WASTE / ADJUST)

suppliers

Supplier directory with lead times and contact info

`purchase_orders`

POs with status (DRAFT!'SENT!'CONFIRMED!'RECEIVED)

`lab_sessions`

AI Kitchen Lab conversations — brief, messages, output recipe

7. Implementation Roadmap

1	Foundation — Auth, Recipes, DB Schema, Base UI ' COMPLETED — Weeks 1–4 (COMPLETED)
2	Operations Core — POS, Kitchen Stock, FOH Stock Weeks 5–10
3	Intelligence Layer — Auto-Ordering, Supplier Portal Weeks 11–15
4	AI Kitchen Lab — Claude Integration, Chef Collab Tools Weeks 16–20
5	Analytics & Polish — Dashboards, Reports, Mobile QA Weeks 21–24

Timeline Summary

Phase	Deliverable	Target
1 (Done)	Auth, Recipes, Ingredients, Base UI	Week 4
2	POS, KDS, Kitchen & FOH Stock	Week 10
3	Supplier auto-ordering, PO workflow	Week 15
4	AI Kitchen Lab (Claude API)	Week 20
5	Analytics, Reports, Mobile optimization	Week 24

8. Security Design

Concern	Solution
Authentication	JWT sessions via NextAuth v5, HttpOnly cookies
Authorization	Role-based middleware on every API route
SQL Injection	Prisma parameterised queries — no raw SQL
XSS	React escapes by default; CSP headers via Next.js
Secrets	Environment variables only, never committed to git
POS Integrity	Server-side price validation — client never sets price
Audit Trail	Every stock/order mutation logged with user + timestamp

9. User Roles & Permissions

Role	Primary Access
ADMIN	Full system access, user management, all settings
MANAGER	All ops: approve POs, view analytics, manage menu
CHEF	Recipes, Kitchen Stock, Kitchen Lab, KDS
SOUS_CHEF	Recipes (read/update), Kitchen Stock, KDS
WAITER	POS only — create orders, mark items served
BARTENDER	POS (bar tab), FOH Stock

